Informe de la Aplicaci?n

Proyecto: Soporte T?cnico / Autogesti?n

Archivo analizado: proyecto_soporte_tecnico.py

Tecnolog?a principal: Reflex + Python + componentes reactivamente actualizados

Resumen ejecutivo

La aplicaci?n es un sistema de soporte t?cnico y gesti?n de incidencias orientado a una instituci?n o municipio. Est? implementada como una app web con Reflex (Python), donde toda la l?gica de negocio y la interfaz gr?fica se resuelven dentro de un mismo archivo principal. El flujo principal consiste en autenticaci?n, visualizaci?n de tickets, gesti?n de inventario, reportes y administraci?n de usuarios.

Estructura interna del c?digo

? Secci?n 1: Constantes y configuraci?n (DIRECCIONES).

? Secci?n 2: Estado global de la aplicaci?n (clase State).

? Secci?n 3: Componentes y vistas UI (funciones que devuelven componentes Reflex).

? Secci?n 4: Configuraci?n de la app (app = rx.App() y route principal).

? Dise?o arquitect?nico: monolito funcional en un solo archivo, no modularizado por paquetes.

1. Estado global y manejo de datos

La clase State hereda de rx.State y concentra la l?gica reactiva. Gu?a el estado de sesi?n, login, roles, tickets, inventario, historial de chat, filtros, paginaci?n y vistas. Esta clase centraliza el estado de toda la aplicaci?n y hace que cada componente reaccione a cambios sin necesidad de un estado externo.

? logged_in, usuario_actual, direccion_actual, es_admin: control de autenticaci?n y sesi?n.

? usuarios_registrados: usuarios base del sistema, incluyendo administrador y solicitante por defecto.

? inventario: lista en memoria con componentes de ejemplo.

? tickets: almacenamiento local de solicitudes con atributos como estado, prioridad, direcci?n, mensaje, nota t?cnica.

? historial_chat: conversaci?n del usuario con soporte t?cnico.

? menu_activo, sidebar_colapsado, pestana_chat_activa: configuraci?n de navegaci?n.

? busqueda, filtro_estado, filtro_direccion, pagina_actual: filtros de gesti?n y paginaci?n.

2. Autenticaci?n y control de acceso

La autenticaci?n se realiza en memoria con validaci?n directa sobre la lista usuarios_registrados. La clase incluye m?todos como iniciar_sesion, registrar_usuario, recuperar_contrasena, cerrar_sesion y

cambiar_rol_usuario.

? Inicio de sesi?n con validaci?n de usuario y contrase?a.

? Registro de nuevos usuarios con nombre, usuario, contrase?a y direcci?n.

? Recuperaci?n de contrase?a seg?n nombre de usuario.

? Cambio de rol administrador/solicitante.

3. Gesti?n de tickets y soporte t?cnico

Los tickets se almacenan como diccionarios dentro de la propiedad tickets. Se utilizan para representar solicitudes del usuario, su direcci?n, prioridad, estado, fecha y nota t?cnica del soporte.

? cargar_datos_ejemplo(): crea datos iniciales ficticios para probar la aplicaci?n.

? enviar_mensaje_chat(): crea un nuevo mensaje y registro de ticket a partir del chat del usuario.

? cambiar_estado_ticket(): administra el cambio de estado del ticket por parte del administrador.

? guardar_nota_tecnica(): permite dejar un informe t?cnico asociado al ticket.

? ver_detalle_ticket(): abre un modal con informaci?n detallada e impresi?n del ticket.

4. Dashboard y reportes

La aplicaci?n ofrece una vista de panel con m?tricas cuantitativas y estad?sticas de proceso. Se calculan variables derivadas, como total_tickets, total_en_proceso, total_en_progreso, total_hechos, tasa_solucion y datos_grafico_estados.

? Tarjetas estad?sticas para total de solicitudes y estados.

? Filtros por todos, mis tickets, en proceso, en progreso y hechos.

? B?squeda por usuario, asunto o mensaje.

? Paginaci?n para la visualizaci?n de tickets.

? Gr?fico de barras para distribuir solicitudes por estado.

? Resumen ejecutivo con tasa de soluci?n efectiva.

5. Inventario

La vista de inventario permite agregar nuevos componentes con c?digo, nombre, categor?a, cantidad y estado. Los datos se gestionan tambi?n en memoria.

? Formularios para agregar nuevos art?culos.

? Tabla con c?digo, nombre, categor?a, stock y estado.

? Valores iniciales de ejemplo cargados al iniciar sesi?n.

6. Vistas principales

? vista_autenticacion(): pantalla de login/registro/recuperaci?n de contrase?a.

? panel_lateral_sidebar(): barra de navegaci?n lateral colapsable.

? vista_chat_modulo(): chat con historial, nueva solicitud y detalle de ?ltima incidencia.

? vista_dashboard_tickets(): gesti?n general de tickets.

? vista_reportes_analitica(): m?dulo de reportes y KPI.

? vista_admin_usuarios(): administraci?n de usuarios y permisos.

? vista_inventario(): gesti?n del stock y equipos.

? modal_impresion_ticket(): detalle de ticket e impresi?n.

7. Navegaci?n y estado visual

La navegaci?n se maneja a trav?s de la propiedad menu_activo y un match de estado en layout_principal. Dependiendo del men? seleccionado, se renderiza una vista distinta. La app activa la vista principal solo cuando el usuario inicia sesi?n.

8. Observaciones t?cnicas

? La soluci?n est? implementada como monolito en un solo archivo, lo cual facilita desarrollo r?pido pero dificulta mantenimiento a mediano plazo.

? La persistencia no existe; todos los datos viven en memoria durante la ejecuci?n, por lo que reiniciar la aplicaci?n elimina registros.

? La l?gica de usuarios y tickets se basa en listas en memoria y no usa autenticaci?n segura ni cifrado de contrase?as.

? La app cuenta con un flujo de negocio ?til para un prototipo funcional y demostraci?n operativa.

? El archivo incluye la l?gica de UX y backend en una sola capa, lo que se puede refactorizar a microservicios o m?dulos con el tiempo.

9. Conclusi?n

La aplicaci?n es una soluci?n de soporte t?cnico y gesti?n operativa construida sobre Reflex, con enfoque de escritorio/web orientado a oficinas o entidades p?blicas. Tiene una estructura funcional clara, aunque a?n no madura a una arquitectura con base de datos, servicios separados y reglas de seguridad m?s robustas. Es adecuada como prototipo, demo o herramienta interna de gesti?n.

Generado autom?ticamente a partir del an?lisis del c?digo fuente.